

Interpretable NLG for Task-oriented Dialogue Systems with Heterogeneous Rendering Machines

Yangming Li^{1,2*}, Kaisheng Yao²

¹Harbin Institute of Technology

²Ant Group

yangmingli@ir.hit.edu.cn, kaisheng.yao@antgroup.com

Abstract

End-to-end neural networks have achieved promising performances in natural language generation (NLG). However, they are treated as black boxes and lack interpretability. To address this problem, we propose a novel framework, heterogeneous rendering machines (HRM), that interprets how neural generators render an input dialogue act (DA) into an utterance. HRM consists of a renderer set and a mode switcher. The renderer set contains multiple decoders that vary in both structure and functionality. For every generation step, the mode switcher selects an appropriate decoder from the renderer set to generate an item (a word or a phrase). To verify the effectiveness of our method, we have conducted extensive experiments on 5 benchmark datasets. In terms of automatic metrics (e.g., BLEU), our model is competitive with the current state-of-the-art method. The qualitative analysis shows that our model can interpret the rendering process of neural generators well. Human evaluation also confirms the interpretability of our proposed approach.

1 Introduction

Natural language generation (NLG), a critical component of task-oriented dialogue systems, converts a meaning representation, i.e., dialogue act (DA), into a natural language utterance. As demonstrated in Figure ??, an input DA consists of an act type and a set of slot values, while the ground truth is a sequence of words. We categorize all the slots into three types: 1) delexicalizable slot whose value is always propagated verbatim to the utterance (colored in red); 2) indicative slot whose value is yes, no, and so on (blue); 3) reworded slot whose value is possibly paraphrased into another expression in the utterance (green). Note that the slot types are defined by us for better understanding and this information is not available in the datasets.

Conventional approaches (??) are mostly pipeline-based, dividing NLG tasks into sentence planning and surface realization. Sentence planning determines a tree-like structure of the given input DA, while surface realization linearizes the structure into the final surface form. Although these models are of great interpretability, they heavily rely on handcraft

Figure 1: This example case is extracted from E2E-NLG dataset (?). The upper part is a DA and the lower part is an utterance.

rules and domain-specific knowledge. Recently, data-driven methods using end-to-end neural networks have attracted much attention (???). For example, ? apply sequence-to-sequence learning (?) to model response generation. These methods facilitate end-to-end learning on the unaligned corpus and achieve state-of-the-art results. However, they are treated as black boxes and lack interpretability. In this work, we consider a neural generator is interpretable if it’s capable of showing how slot values are rendered and their locations in the generated utterance.

To generalize NLG models to low-frequency and unseen slot values, prior methods mostly adopt the delexicalization technique, where slot values in the utterance are replaced with corresponding placeholders (???). While delexicalization, to some extent, has improved the interpretability, it’s applicable for delexicalizable slots only, and therefore has limited usage. This is especially apparent in datasets with the majority of slots being reworded, e.g., E2E-NLG dataset (?). Moreover, ?? observe that using delexicalization can result in lexical errors. Visualizing attention weight is another approach to understanding neural generators. However, many works (?) have found that it’s hard to reach meaningful interpretability.

In this work, we present a novel framework, heterogeneous rendering machines (HRM), that is both end-to-end and interpretable. Through using HRM as the decoder, we can interpret the mapping from input a DA to an utterance. HRM consists of a renderer set and a mode switcher. The renderer set is a collection of multiple decoders that vary in structure and functionality. In implementation, we set a conditional sequence generator (???) to render indicative slots and reworded slots, a pointer network (?) to explicitly copy the values of delexicalizable slots, and a language model (??) to produce context-agnostic words. The mode switcher is a discrete latent variable that selects a proper decoder from the renderer set for every generation step. We explore using two techniques, Gumbel-softmax (?) and VQ-VAE (?), for the selection. Furthermore, a hierarchical en-

*This work was done when the first author did internship at Ant Group.

Copyright © 2021, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

Figure 2: The architecture of the proposed model. Different modules are differentiated by colors.

coder is adopted to represent DA as a collection of coarse-grained semantic units. We use a slot-level LSTM (?) to encode slot values and a DA-level self-attention mechanism (?) to capture the semantic correlations among them.

Extensive experiments are conducted on multiple benchmark datasets. In terms of automatic metrics, our model is competitive with the existing state-of-the-art method. Qualitative analysis shows that our model is capable of interpreting the rendering process well. We also conduct human evaluation to confirm the effectiveness of our method.

2 Architecture

Figure ?? demonstrates our model. Firstly, a hierarchical encoder embeds input DA into a set of semantic units. Then, for each generation iteration, mode switcher aggregates the predictions made by different renderers.

Formally, we represent an input DA as a n -sized set $\mathbf{x} = \{(s_1, \mathbf{v}_1), (s_2, \mathbf{v}_2), \dots, (s_n, \mathbf{v}_n)\}$. Here s_1 is the act type and v_1 is a dummy symbol "TYPE". Other elements $(s_i, \mathbf{v}_i), i > 1$ are the slot value pairs. The ground truth utterance is a m -length list $\mathbf{y} = [y_1, y_2, \dots, y_m]$. Every item in it is either a word or a phrase (i.e., slot value). For instance, the case in Figure ?? is represented as

$$\begin{aligned} \mathbf{x} &= \{(\text{inform}, [\text{TYPE}]), (\text{name}, [\text{Blue}, \text{Spice}]), \\ &\quad (\text{familyFriendly}, [\text{yes}]), \\ &\quad (\text{priceRange}, [\text{less}, \text{than}, 20])\} \\ \mathbf{y} &= [\text{"The"}, \text{"Blue Spice"}, \text{"is"}, \text{"a"}, \text{"low"}, \\ &\quad \text{"cost"}, \text{"venue"}, \text{"."}, \text{"it's"}, \text{"a"}, \\ &\quad \text{"family"}, \text{"friendly"}, \text{"location"}, \text{"."}] \end{aligned}$$

Originally, the output utterance is a sequence of words. The above format is obtained by merging successive words that match some input slot value (delexicalizable or reworded) verbatim into a phrase.

2.1 Hierarchical Encoder

Hierarchical encoder converts input DA into a collection of coarse-grained semantic units.

Firstly, a slot-level LSTM f^s is used to encode each value $\mathbf{v}_i = [v_{i,1}, v_{i,2}, \dots, v_{i,l_i}]$ as

$$\begin{cases} \vec{\mathbf{h}}_{i,k}^v = \vec{f}^s(\vec{\mathbf{h}}_{i,k-1}^v, \mathbf{E}(v_{i,k})) \\ \overleftarrow{\mathbf{h}}_{i,k}^v = \overleftarrow{f}^s(\overleftarrow{\mathbf{h}}_{i,k+1}^v, \mathbf{E}(v_{i,k})) \\ \mathbf{h}_{i,k}^v = \vec{\mathbf{h}}_{i,k}^v \oplus \overleftarrow{\mathbf{h}}_{i,k}^v \end{cases} \quad (1)$$

in which $\mathbf{E}$ is the word embedding.

Then, we embed every slots s_i in DA $\mathbf{x}$:

$$\mathbf{h}_i^s = \mathbf{E}(s_i). \quad (2)$$

We represent each slot value pair by column-wise vector concatenation as $\mathbf{h}_i^d = \mathbf{h}_i^s \oplus \mathbf{h}_{i,l_i}^v$. In training, a word is randomly masked with "UNK" with probability $1/(1+p)$

(p is its frequency in the training set). This helps generalize the models to unseen values in the test set.

Eventually, we leverage DA-level self-attention mechanism (?) to capture the semantic correlations among slot values as

$$\begin{cases} \mathbf{H}^d = [\mathbf{h}_1^d; \mathbf{h}_2^d; \dots; \mathbf{h}_n^d] \\ \mathbf{Q}^d, \mathbf{K}^d, \mathbf{V}^d = \mathbf{W}_q \mathbf{H}^d, \mathbf{W}_k \mathbf{H}^d, \mathbf{W}_v \mathbf{H}^d \\ \mathbf{H}^e = \text{Softmax}(\frac{\mathbf{Q}^d (\mathbf{K}^d)^T}{\sqrt{d}}) \mathbf{V}^d, \\ \mathbf{H}^e = [\mathbf{h}_1^e; \mathbf{h}_2^e; \dots; \mathbf{h}_n^e] \end{cases} \quad (3)$$

where $[\cdot]$ means row-wise vector concatenation and d is the dimension of hidden layers.

Prior methods mostly adopt LSTM or one-hot encoding as encoders. However, LSTM (?) is order-sensitive and reads DA sequentially, ignoring the logic structure. One-hot encoding can't handle reworded slots. Our encoder circumvents all these issues.

2.2 Heterogeneous Rendering Machines

HRM is the decoder of our proposed model, which aims to interpret how neural generators render input DA into an utterance. It consists of two components, including a renderer set and a mode switcher.

Renderer Set Renderer set is a collection of decoders varying in both structure and functionality. In the proposed model, we set a pointer network, a conditional sequence generator, and a language model.

In this section, we assume that it's the j -th generation step. The decoder hidden state and the generated item of prior step are denoted as $\mathbf{h}_{j-1}^d$ and $\mathbf{y}_{j-1}$, respectively. Note that $\mathbf{y}_{j-1}$ is embedded as $\mathbf{h}_i^s$ if it's a phrase directly copied from the value $\mathbf{v}_i$.

Pointer Network explicitly copies the slot value that exactly matches successive words in the utterance. Firstly, an LSTM g^p is used to update the hidden state:

$$\mathbf{h}_j^p = g^p(\mathbf{h}_{j-1}^d, \mathbf{E}(\mathbf{y}_{j-1})). \quad (4)$$

Then, analogous to pointer network, we apply soft attention over input slot value pairs as

$$\begin{cases} \beta_{j,i} = \mathbf{v}_\beta^T \tanh(\mathbf{W}_\beta \mathbf{h}_j^p + \mathbf{U}_\beta \mathbf{h}_i^e) \\ \mathbf{Q}_j^p = \text{Softmax}([\beta_{j,1}, \beta_{j,2}, \dots, \beta_{j,n}]) \end{cases} \quad (5)$$

where $\mathbf{v}_\beta$, $\mathbf{W}_\beta$, and $\mathbf{U}_\beta$ are learnable.

Conditional Sequence Generator renders some slot value into various forms in the sentence. Firstly, we apply attention mechanism over the encoded DA representations:

$$\begin{cases} \alpha_{j,i} = \mathbf{v}_\alpha^T \tanh(\mathbf{W}_\alpha \mathbf{h}_{j-1}^d + \mathbf{U}_\alpha \mathbf{h}_i^e) \\ \alpha'_{j,i} = \frac{\exp(\alpha_{j,i})}{\sum_{1 \leq k \leq n} \exp(\alpha_{j,k})} \\ \mathbf{h}_j^a = \sum_{1 \leq i \leq n} \alpha'_{j,i} \mathbf{h}_i^e \end{cases} \quad (6)$$

Figure 3: Two attempts to implement the mode switcher: Gumbel-softmax (left) and VQ-VAE (right).

Then, we employ a LSTM g^c to read prior item and update the decoder hidden state as

$$\mathbf{h}_j^c = g^c(\mathbf{h}_{j-1}^d, \mathbf{E}(y_{j-1}) \oplus \mathbf{h}_j^a). \quad (7)$$

Finally, it predicts the next word distribution as

$$\mathbf{Q}_j^c = \mathbf{W}_c(\mathbf{h}_j^c \oplus \mathbf{h}_j^a), \quad (8)$$

where $\mathbf{W}_c$ is also a learnable parameter.

Unconditional Language Modeling produces context-agnostic words (e.g., “is” and “a”) in Figure ?? . Firstly, we apply an LSTM g^l to read the previous item as

$$\mathbf{h}_j^l = g^l(\mathbf{h}_{j-1}^d, \mathbf{E}(y_{j-1})). \quad (9)$$

Then, we predict the next word distribution:

$$\mathbf{Q}_j^l = \text{Softmax}(\mathbf{W}_l \mathbf{h}_j^l). \quad (10)$$

$\mathbf{Q}_j^c$ and $\mathbf{Q}_j^l$ are the distributions over predefined word vocabulary, while $\mathbf{Q}_j^p$ is the one over input slot values.

Mode Switcher Mode switcher is a discrete latent variable. At each generation step, its aim is to choose an appropriate decoder from the renderer set. To implement it, we explore using Gumbel-softmax trick and further introduce a variant of VQ-VAE.

The output $\mathbf{o}_j = [o_j^p, o_j^c, o_j^l]$ of it is an one-hot vector. We use it to aggregate the updated hidden states and the predicted next word distributions of different renderers:

$$\begin{cases} \mathbf{h}_j^d = o_j^p \mathbf{h}_j^p + o_j^c \mathbf{h}_j^c + o_j^l \mathbf{h}_j^l \\ \mathbf{Q}_j = o_j^p \mathbf{Q}_j^p + o_j^c \mathbf{Q}_j^c + o_j^l \mathbf{Q}_j^l \end{cases} \quad (11)$$

At test time, we make the next word prediction as $\hat{y}_j = \arg \max \mathbf{Q}_j$. In the rest of this section, we describe two approaches to implement it.

Gumbel-softmax Trick (?) solve the nondifferentiability problem of sampling from a categorical distribution. The main idea is using the differentiable sample from a gumbel-softmax distribution which can be smoothly annealed into given categorical distribution.

The procedure is demonstrated in the left part of Figure ?? . Firstly, we predict a categorical distribution $\mathbf{z}_j$ over the renderer set as

$$\begin{cases} [\theta_j^p, \theta_j^c, \theta_j^l] = \mathbf{W}_\theta \tanh(\mathbf{U}_\theta \mathbf{h}_{j-1}^d) \\ [z_j^p, z_j^c, z_j^l] = \text{Softmax}([\theta_j^p, \theta_j^c, \theta_j^l]) \end{cases} \quad (12)$$

Then, the following operation is used as a differentiable approximation to sampling:

$$o_j^r = \frac{\exp((\log(z_j^r) + \gamma^r)/\tau)}{\sum_{r' \in \{p, c, l\}} \exp((\log(z_j^{r'}) + \gamma^{r'})/\tau)}, \quad (13)$$

where subscript r iterates over $\{p, c, l\}$. γ^p , γ^c , and γ^l are i.i.d samples drawn from Gumbel(0, 1) (?).

VQ-VAE (?) use vector-quantized variational autoencoder (VQ-VAE) to learn discrete representations. The recognition network outputs discrete codes and its parameters are optimized by straight-through estimator (?). In implementations, we have made two changes: 1) reparameterization trick (?) is used to add randomness; 2) the codebook (i.e., embedding space in the original paper) is dynamic rather than static.

As depicted in the right part of Figure ?? , firstly, we apply reparameterization trick on hidden state $\mathbf{h}_{j-1}^d$ to get a stochastic variable $\mathbf{h}_j^v$ as

$$\begin{cases} \mu_j = \mathbf{W}_\mu \tanh(\mathbf{U}_\mu \mathbf{h}_{j-1}^d) \\ \sigma_j = \mathbf{W}_\sigma \tanh(\mathbf{U}_\sigma \mathbf{h}_{j-1}^d) \\ \mathbf{h}_j^v = \mu_j + \epsilon \odot \exp(\sigma_j) \end{cases} \quad (14)$$

where ϵ is sampled from standard Gaussian distribution and $\odot$ is element-wise product.

Then, we quantize the variable $\mathbf{h}_j^v$ in terms of the dynamic codebook that contains different decoder hidden states:

$$\begin{cases} z_j^r = \|\mathbf{h}_j^v - \mathbf{h}_j^r\|_2, r \in \{p, c, l\} \\ r' = \arg \min_{r \in \{p, c, l\}} z_j^r \\ [o_j^p, o_j^c, o_j^l] = \text{OneHot}(r') \end{cases} \quad (15)$$

where $\|\cdot\|_2$ is Euclidian distance. Function OneHot constructs a 3-length one-hot vector, where the r' -th value is 1. Following VQ-VAE, we use straight-through estimator to approximate the gradient of arg min.

2.3 Discussion

In this section, we show the rationality of HRM and how it interprets neural generators.

Rationality. We utilize probability theory to analyze the essence of HRM. Generally, our decoder decomposes the joint probability of producing a natural language sentence $\mathbf{y}$ into the ordered conditionals:

$$P(\mathbf{y}|\mathbf{x}) = \prod_{1 \leq i \leq m} P(y_i | y_{<i}, \mathbf{x}). \quad (16)$$

Since an item is produced by either copying, conditional generation, or context-agnostic generation, its probability can be decomposed as

$$\begin{aligned} P(y_i | y_{<i}, \mathbf{x}) = \\ \sum_{r \in \{p, c, l\}} P(y_i | y_{<i}, \mathbf{x}, r) P(r | y_{<i}, \mathbf{x}), \end{aligned} \quad (17)$$

which corresponds to Equation ?? . $P(r | y_{<i}, \mathbf{x})$, $r \in \{p, c, l\}$ is the output of mode switcher. $P(y_i | y_{<i}, \mathbf{x}, r)$ is the next item distribution predicted by corresponding renderer. For example, l represents language model.

To permit interpretability, mode switcher is modeled as a discrete latent variable. In Equation ?? , it means that distribution $P(r | y_{<i}, \mathbf{x})$ is “one hot”. However, experiments show that this degrades the performances.

Model		Restaurant		Hotel		Laptop		Television		E2E-NLG
		BLEU	ERR	BLEU	ERR	BLEU	ERR	BLEU	ERR	BLEU
HLSTM (?)		0.747	0.74	0.850	2.67	0.513	1.10	0.525	2.50	-
SCLSTM (?)		0.753	0.38	0.848	3.07	0.512	0.79	0.527	2.31	-
TGen (?)		-	-	-	-	-	-	-	-	0.659
RALSTM (?)		0.779	0.16	0.898	0.43	0.525	0.42	0.541	0.63	-
NLG-LM (?)		0.795	-	0.939	-	0.586	-	0.617	-	0.684
Our Model	Gumbel-softmax	0.776	0.21	0.903	0.77	0.561	0.63	0.581	0.79	0.667
	VQ-VAE	0.789	0.18	0.921	0.55	0.554	0.65	0.598	0.70	0.681
	Softmax	0.812	0.20	0.923	0.46	0.591	0.31	0.610	0.51	0.697

Table 1: Experiment results on five datasets for all baselines and our models.

	R	H	L	T	E
training set	3114	3223	7944	4221	42061
validation set	1039	1075	2649	1407	4672
test set	1039	1075	2649	1407	4693
total	5192	5373	13242	7035	47366
DA types	8	8	14	14	1
slot types	12	12	20	16	8

Table 2: The details of different datasets.

Interpretability. We can understand how neural generators render input DA into an utterance using the following procedure. For each item y_j in the generated sentence, the output $\mathbf{o}_j$ of mode switcher indicates which renderer produces it. If the renderer is pointer network, pointer attention $\mathbf{Q}_j^p$ shows where the item is copied from. If it's conditional sequence generator, soft attention $[\alpha'_{j,1}, \alpha'_{j,1}, \dots, \alpha'_{j,n}]$ indicates which slot value the item is reworded from. Otherwise, the item is a context-agnostic word generated by the language model.

3 Training Criteria

The following cross entropy loss is incurred for training:

$$L^c = \sum_{1 \leq j \leq m} -\log \mathbf{Q}_j[y_j]. \quad (18)$$

If mode switcher is implemented as VQ-VAE, we add two extra criterions. Firstly, following ?, we have:

$$L^d = \sum_{1 \leq j \leq m} \|\text{sg}[\mathbf{h}_j^v] - \mathbf{h}_j^{r'}\|_2^2 + \rho \|\mathbf{h}_j^v - \text{sg}[\mathbf{h}_j^{r'}]\|_2^2, \quad (19)$$

where sg stands for the stopgradient operator that is defined as identity at forward computation time and has zero partial derivatives. In all experiments, we set ρ as 0.25. Secondly, following ?, we adopt KL divergence to avoid posterior collapse:

$$L^v = \sum_{1 \leq j \leq m} (\mu_j^2 + \exp(\sigma_j) - (1 + \sigma_j)). \quad (20)$$

If using Gumbel-softmax, the annealing schedule for τ is the same as that in ?.

4 Experiments

To verify the effectiveness of our model, we have conducted comprehensive studies on five datasets. The main experiments show that our model is competitive with the current state-of-the-art approach. Case studies demonstrate that the proposed model can interpret the rendering process well. We also investigate ablation experiments to explore the impacts of some components.

4.1 Settings

We evaluate the models on five benchmark datasets. The Hotel dataset and the Restaurant dataset are collected in (?). The Laptop dataset and the TV dataset are from (?). The E2E-NLG dataset is released by a shared challenge (?)¹. All the datasets used in our paper follow the same format, pretreatment, and partition as in (???). Other details of the datasets are demonstrated in Table ??.

For fairness, we use the official evaluation scripts from repositories: E2E-NLG² and RNN-LG³. The automatic metrics include BLEU and slot error rate (ERR). ERR is computed as $\text{ERR} = \frac{p+q}{N}$, where N is the total number of slots in the DA, and p, q is the amount of missing and redundant slots in the generated utterance, respectively.

We adopt the same configurations for all five datasets. The dimensionalities for all embeddings are 256. The hidden units of all layers are set as 512. We adopt 3 layers of self-attention and each of them has 4 heads. L2 regularization is set as 1×10^{-6} and the dropout ratio is assigned 0.4 for reducing overfit. Above setting is obtained by using grid search. We use Adam (?) to optimize model parameters. All the studies are conducted at GeForce RTX 2080T. For each DA, we over-generate 10 utterances through beam search and select the top 5 candidates. In experiments, we select the model that works the best on the dev set, and then evaluate it on the test set. The improvements of our models over the baselines are statistically significant with $p < 0.05$ under t-test.

4.2 Main Results

The baselines for comparison are as follows:

¹<http://www.macs.hw.ac.uk/InteractionLab/E2E/>.

²<https://github.com/tuetschek/e2e-metrics>.

³<https://github.com/shawnwun/RNNLG>.

Figure 4: The cases are extracted from E2E-NLG dataset to show the interpretability of HRM.

- HLSTM (?) designs a heuristic gate to control the states of slot values, guaranteeing that all of them are accurately captured;
- SCLSTM (?) introduces an extra “reading gate to improve standard LSTM;
- TGen (?) uses encoder-decoder architecture augmented with attention mechanism to generate utterances from input DA;
- RALSTM (?) presents a RNN-based decoder to select and aggregate the semantic elements produced by attention mechanism;
- NLG-LM ? incorporates language modeling into response generation. In particular, it provides current state-of-the-art performances on BLEU score for all the datasets.

We follow the baseline results as reported in ??.

Table ?? presents main results. First of all, VQ-VAE is more effective than gumbel-softmax for modeling mode switcher. Except for Laptop dataset, using VQ-VAE obtains better performances than using gumbel-softmax. In terms of BLEU scores, it improves the performances by 1.68% on Restaurant dataset, 1.99% on Hotel dataset, 2.84% on Television dataset, and 2.06% on E2E-NLG dataset. For correctness, it also reduces the ERR by 14.29% on Restaurant dataset, 28.57% on Hotel dataset, and 11.39% on Television dataset. Secondly, our model is not only interpretable but also competitive with current state-of-the-art method. For instance, our BLEU scores are lower than those of NLG-LM by only 0.75% on Restaurant dataset and 0.44% on E2E-NLG dataset.

Lastly, we discover that there is a trade-off between Interpretability and effectiveness. The results shown in the last row are obtained by directly using the categorical distribution $[z_j^p, z_j^c, z_j^l]$ predicted in Equation ?? as the output $\mathbf{o}_j$ of mode switcher. Although it degrades the interpretability, the performances are notably improved. Compared with NLG-LM, it achieves the increasements on BLEU scores of 2.14% on Restaurant dataset, 0.85% on Laptop dataset, and 1.90% on E2E-NLG dataset. For ERR, the scores are also reduced by 26.19% on Laptop dataset and 19.05% on Television dataset. We guess that using discrete representations limits the flexibility of neural networks.

4.3 Case Study

In this section, we show that HRM is capable of interpreting the rendering process well.

Using the procedure described in Section ??, we label a generated item as $P(i)$ if it’s copied from the i -th slot value, $C(i)$ if it’s reworded from the i -th slot value, or L otherwise. Several examples are depicted in Figure ??, where blue lines and red lines denote paraphrasing and copying, respectively. In most of the cases, HRM is of great interpretability. For example, in the third case, phrase “has low customer ratings” is aligned with value “1 out of 5”. In the second case,

Method	Hotel	E2E-NLG
Our Model	0.921	0.681
w/o Self-attention	0.908	0.651
w/o Self-attention, w/ LSTM	0.914	0.672
w/o Pointer Net	0.862	0.665
w/o Pointer Net, w/ Delex.	0.910	0.669

Table 3: Ablation experiments on two datasets.

Method	E2E-NLG
TGen	0.562
NLG-LM	0.495
Our Model w/ Gumbel-softmax	0.815
Our Model w/ VQ-VAE	0.872
Our Model w/ Softmax	0.621

Table 4: Human evaluation on E2E-NLG dataset.

phrase “an adult restaurant” is aligned with slot value (“familyFriendly”, “no”). There also exists a few mistakes. For instance, in the fourth case, phrase “outside Cambridge City Centre” is not tied with value “riverside”. We attribute this to the high fault tolerance of end-to-end neural networks. In addition, here we omit act type for brevity.

4.4 Ablation Study

As shown in Table ??, we conduct ablation study to explore the impacts of components.

Effect of DA-level Encoder. By removing self-attention, the BLEU scores sharply drop by 1.41% and 4.41%. Hence it’s useful to capture the semantic correlations among slot values. Additionally, replacing self-attention with LSTM decreases the BLEU scores by 0.76% and 1.32%. We credit this to the order-insensitivity of self-attention.

Effect of Copying Mechanism. By removing pointer network, the BLEU scores decrease by 6.41% and 2.35%. Hence directly copying words from input DA is useful. Replacing pointer network with delexicalization reduces the BLEU scores by 1.19% and 1.76%. Besides, delexicalization can only apply to delexicalizable slots. These indicate that pointer network is more effective.

4.5 Human Evaluation

We conduct human evaluation to quantitatively make comparisons on interpretability among different models. The task consists of three stages. Firstly, we introduce an alignment score to assess the interpretability of a model. It’s computed as p/N . p is the number of slot values that are correctly aligned with the utterance by a model (see Figure ??). N is the total amount of slot values. Then, we define how various generators interpret their rendering processes. For our

models, we do interpretation as described in Section ?? . For the baselines (e.g., TGen), we use the computed attention weights to align the input DA with the generated sentence. Eventually, we randomly sample 200 cases from the test set as the benchmark set and recruit 15 people to judge whether a slot value is aligned well with the generated utterance (i.e., getting p). The designed task is very simple and highly consistent among workers.

Table ?? demonstrates the experiment results. From it, we can draw the following conclusions. Firstly, better performances may lead to worse interpretability. For example, the state-of-the-art model, NLG-LM, underperforms a simple baseline, TGen, by 13.5%. From Table ?? , we can see that, for our model, using softmax generally obtains higher F1 scores than adopting discrete latent variable models. However, here we find the interpretability score of using softmax is lower than using VQ-VAE or Gumbel-softmax. Secondly, our models consistently and significantly outperform prior baselines. For example, the F1 score of Our Mode w/ VQ-VAE outnumbers TGen by 35.6% and NLG-LM by 43.2%. Thirdly, VQ-VAE is better than Gumbel-softmax in terms of both BLEU score and interpretability score. For example, in Table ?? , using VQ-VAE outperforms using Gumbel-softmax by 1.65% on Restaurant, 2.84% on Television, and 2.06% on E2E-NLG. In Table ?? , the increase percentage of interpretability score is 6.54%.

5 Related Work

In task-oriented dialogue systems, NLG is the final module to produce user-facing system utterances, which is directly related to the perceived quality and usability. Traditional approaches generally divide the NLG task into a pipeline of sentence planning and surface realization (????). Sentence planning first converts an input DA into a tree-like structure, and then surface realization maps the intermediate structure into the final surface form. For example, ? use a class-based n -gram language model and a template-based reranker. ? address the limitations of n -gram language models by using more complex syntactic trees. ? employ a phrase-based generator that learns from a semantically aligned corpus. Although these methods are adequate and of great interpretability, they are heavily dependent on handcraft rules and expert knowledge. Moreover, the sentences generated from rule-based systems are often rigid, without the diversity and naturalness of human language.

Lately, there is a surge of interest in utilizing neural networks to build corpus-based NLG models (????). The main superiority is facilitating end-to-end training on the unaligned corpus. For example, ? present a heuristic gate to guarantee that all slot value pairs are accurately captured during generation. ? introduce a novel SC-LSTM with an additional reading cell to learn gated mechanism and language model jointly. ? use encoder-decoder architecture augmented with attention mechanism to generate utterances from input DA. ? use a RNN-based decoder to select and aggregate the semantic elements produced by attention mechanism. Most recently, ? incorporate a language model task into the response generation process to boost the naturalness of generated utterances. ? study the slot consistency is-

sue and propose a novel iterative rectification network to address it. While plenty of state-of-the-art performances have been obtained, they are all treated as black boxes, and thus lack interpretability. Delexicalization (???) to some extent raises the interpretability as it directly locates the position of slot values in the utterance. Nevertheless, it is applicable for delexicalizable slots only. In E2E-NLG dataset, most of the slots are reworded or indicative. ? also observe that using delexicalization results in mistakes.

6 Conclusion

In this paper, we present heterogeneous rendering machines (HRM) to improve the interpretability of NLG models. It consists of a renderer set and a mode switcher. The renderer set contains multiple decoders that vary in structure and functionality. The mode switcher is a discrete latent variable that chooses an appropriate decoder from the renderer set in every generation step. Extensive experiments have been conducted on five datasets, demonstrating that our model is competitive with the current state-of-the-art method. Qualitative studies show that our model can interpret the rendering process well. Human evaluation further confirms its effectiveness in interpretability.

Currently, a severe problem in interpretable NLG is lacking a proper evaluation metric. Mainstream metrics such as BLEU are not applicable. Using our alignment score demands massive annotation efforts. We will work hard on this issue in future research.

Acknowledgments

This work was supported by Ant Group through Ant Research Program. We thank anonymous reviewers for their valuable and constructive comments.

Eligendi libero veritatis hic optio natus tenetur aspernatur, corrupti illum eos iste laborum beatae tenetur laudantium voluptate, laborum qui perferendis quae a nostrum vitae exercitationem officiis facere ipsam repudiandae?Dolorum corporis repudiandae repellat ratione